# Lecture 15 & 16 - Module 5.1 Multilayer perceptron

## COMP 551 Applied machine learning

Yue Li
Assistant Professor
School of Computer Science
McGill University

Feb 27 & March 11, 2025

# Outline

# Learning objectives

Understanding the basic ideas of

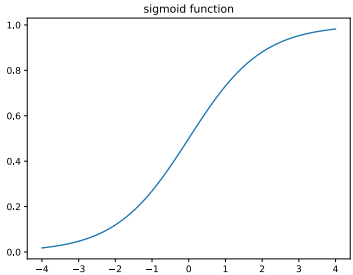
- Perceptron and its limitation
- Multilayer perceptron (MLP)
- Activation functions of MLP
- MLP architecture
- Example applications of MLP

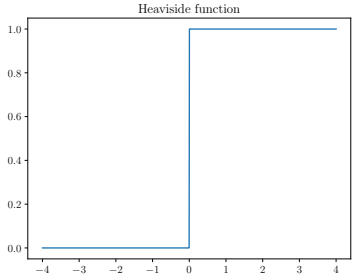
# Outline

# Perceptron – a linear discriminant function

The perceptron was invented in 1943 by McCulloch and Pitts and was first built as a machine by Rosenblatt in 1958. It is a deterministic binary classifier that uses *Heaviside step function* as opposed to sigmoid function $\sigma(\mathbf{xw})$ in the logistic regression:

$$\hat{y}^{(n)} = \mathbb{I}[\mathbf{x}^{(n)}\mathbf{w} + b > 0]$$




Rosenblatt, Frank (1957). "The Perceptron – a perceiving and recognizing automaton". Report 85-460-1. Cornell Aeronautical Laboratory.

# Perceptron learning algorithm

Because the Heaviside function is not differentiable, gradient-based optimization is not suitable. The perceptron learning algorithm will update weights iteratively on *only the misclassified examples*:

$$\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} - \alpha(\hat{y}^{(n)} - y^{(n)})\mathbf{x}^{(n)}$$

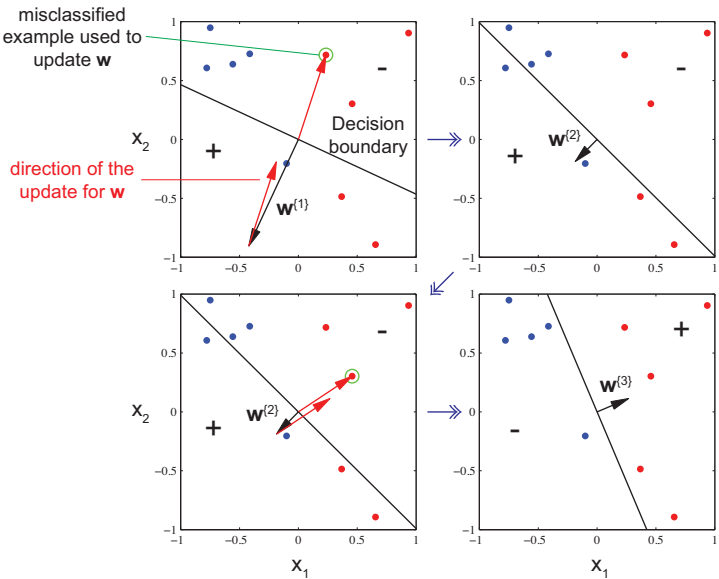
Notice that the update has the same form as the logistic regression gradient but it's updated example and $\hat{y}^{(n)}$ is binary.
By subtracting the gradient from the coefficients, the Perceptron is adjusting its decision boundary to account for each misclassified training example:

- if $y^{(n)} = 1$ and $\hat{y}^{(n)} = 0$, it moves the decision boundary in the direction of the input to include that point as positive $\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} + \mathbf{x}^{(n)}$
- if $y^{(n)} = 0$ and $\hat{y}^{(n)} = 1$, it moves the decision boundary in the opposite direction of the input to exclude that point as positive $\mathbf{w}^{\{t+1\}} \leftarrow \mathbf{w}^{\{t\}} - \mathbf{x}^{(n)}$

# Key Python implementation steps of Perceptron (Colab)

```
t = 0
change = True
max_iters=10000
while change and t < self.max_iters:
    change = False
    for n in np.random.permutation(N):
        yh = (np.dot(x[n,:], w) > 0).astype(int)
        if yh != y[n]:
            w = w - (yh - y[n])*x[n,:]
            change = True
    t += 1
    w_hist.append(w) # record update history
```

# Perceptron learning illustration on 2D input features (Bishop06 Fig. 4.7)

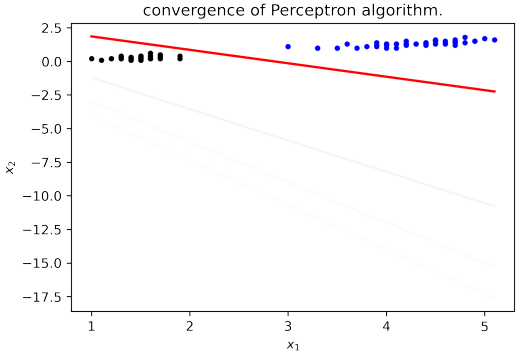



- Perceptron converges on this toy data in two iterations
- However, even data set is linearly separable, there may be many solutions
- The final solution $\hat{\mathbf{w}}$ depends on the initial parameters and the order the training data points the algorithm iterates through

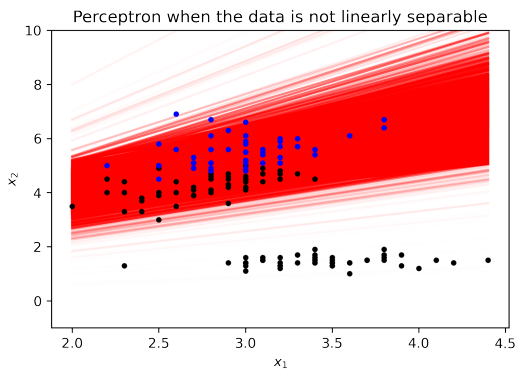
# Perceptron can only converge on linearly separable data (colab)

The decision boundary is when $\mathbf{xw} = 0$. In 2D case, we have $w_0 + x_1 w_1 + x_2 w_2 = 0$. For every value of $x_1$, solve for $x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2} x_1$, which form the decision boundary.

The perceptron converged after 200 iterations after trained on separating Iris flows of class 0 from class 1.




It fails to converge on separating flowers in class 0 from class 1 or 2 because the data are not linearly separable.



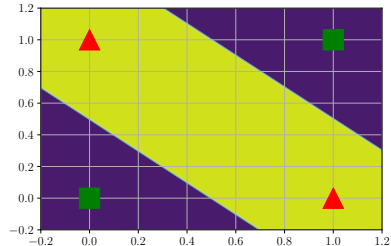
# XOR: the most famous problem that perceptron fails on

Exclusive OR (XOR) function is

$$y = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2) \equiv x_1 \veebar x_2$$

Truth table for the XOR function

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |




With linear decision boundary, Perceptron cannot approximate the XOR function. More limitations of Perceptron was described in Marvin Minsky and Seymour Papert's "infamous" book called *Perceptron* in 1969.

What other approach have we learned that *can* model non-linear functions despite using a linear model?

# Outline

# Composing *L adaptive* basis functions

- Recall in Module 4.1, we can transform features using a set of *fixed* non-linear basis functions. For 1D features, we can have M-degree polynomial features $\Phi(x) = [1, x^1, \ldots, x^M]$ or Gaussian basis functions $\phi_d(x) = \exp\left(-\frac{(x-\mu_d)^2}{2s^2}\right)$
- To make them flexible, we can add *learnable parameters*, $\mathbf{W}^{\{1\}}$ and $\mathbf{W}^{\{2\}}$, to those basis functions:

$$\mathbf{z} = f(\mathbf{x}; \mathbf{W}^{\{1\}}, \mathbf{W}^{\{2\}}) \equiv f_2(f_1(\mathbf{x}\mathbf{W}^{\{1\}})\mathbf{W}^{\{2\}})$$
$$\hat{y} = \mathbf{z}\mathbf{w}^{\{o\}}, \quad J(\mathbf{W}^{\{1\}}, \mathbf{W}^{\{2\}}) = cost(\hat{y}, y)$$

Here, we seek to optimize the coefficients of both functions:

$$\mathbf{W}^{\{1\}}, \mathbf{W}^{\{2\}} \leftarrow \underset{\mathbf{W}^{\{1\}}, \mathbf{W}^{\{2\}}}{\arg\min}\ J(\mathbf{W}^{\{1\}}, \mathbf{W}^{\{2\}})$$

- In general, we can *compose L layers* of functions:

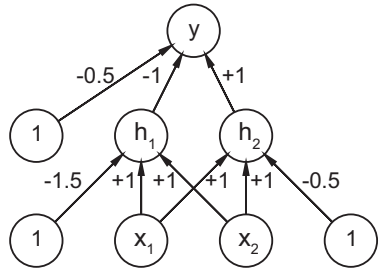
$$\mathbf{z}^{\{L\}} = f_L(f_{L-1}(\ldots f_3(f_2(f_1(\mathbf{x}\mathbf{W}^{\{1\}})\mathbf{W}^{\{2\}})\mathbf{W}^{\{3\}})\ldots\mathbf{W}^{\{L-1\}})\mathbf{W}^{\{L\}})$$

This is the key idea of a Multilayer Perceptron (MLP) (aka Neural Network)

# Modelling the XOR function with MLP

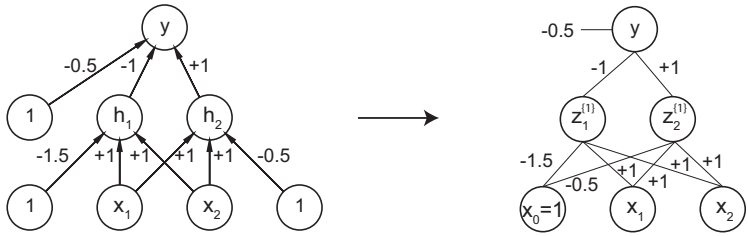
XOR table for $y = \neg(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 1 | 0 |




- Nodes $x_1$ and $x_2$ are called **input units**. Nodes marked as 1 are "always-on" units.
- Nodes $h_1$ and $h_2$ are called **hidden units**. Node $y$ is the **output unit**.
- $h_1 = \mathbb{I}[x_1 + x_2 - 1.5 > 0]$ captures the AND function because it *activates* if both input units are "on", i.e., $x_1 \wedge x_2$
- $h_2 = \mathbb{I}[x_1 + x_2 - 0.5 > 0]$ captures the OR function because it *activates* if one or both are "on", i.e., $x_1 \vee x_2$
- $y = \mathbb{I}[h_2 - h_1 - 0.5 > 0] \equiv \neg h_1 \wedge h_2 = \neg(x_1 \wedge x_2) \wedge (x_1 \vee x_2)$

# MLP notations using XOR-MLP as an example




pre-activated hidden unit 1: $a_1^{\{1\}} = -1.5x_0 + x_1 + x_2 = \mathbf{x}\mathbf{w}_1^{\{1\}}$

activated hidden unit 1: $z_1^{\{1\}} = \mathbb{I}[a_1^{\{1\}} > 0]$

pre-activated unit 2: $a_2^{\{1\}} = -0.5x_0 + x_1 + x_2 = \mathbf{x}\mathbf{w}_2^{\{1\}}$

activated hidden unit 2: $z_2^{\{1\}} = \mathbb{I}[a_2^{\{1\}} > 0]$

output unit: $y = \mathbb{I}[-0.5 - z_1^{\{1\}} + z_2^{\{1\}}]$

Bracketed superscript $^{\{\ell\}}$ indexes $\ell$th layer. The XOR-MLP only has one hidden layer.

# Notations of a 2-layer MLP: layer 1

- $z_j^{\{1\}}$ – hidden units $j \in \{1, 2, 3\}$ at layer 1:

$$a_j^{\{1\}} = \sum_{d=1}^{D=4} w_{d,j}^{\{1\}} x_d = \mathbf{x}\mathbf{w}_j^{\{1\}}$$

$$z_j^{\{1\}} = h(a_j^{\{1\}})$$

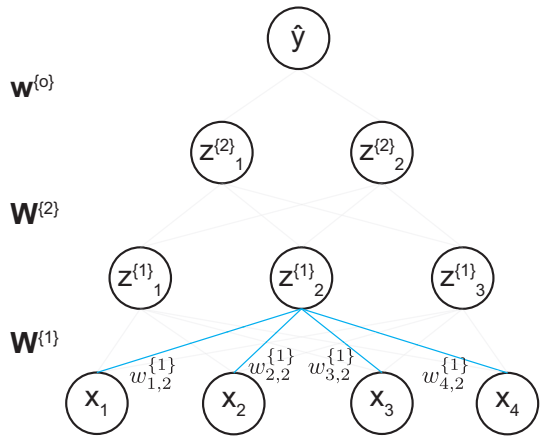
- $z_k^{\{2\}}$ – hidden units $k \in \{1, 2\}$ at layer 2:

$$a_k^{\{2\}} = \sum_{j=1}^{K_1=3} w_{j,k}^{\{2\}} z_j^{\{1\}} = \mathbf{z}^{\{1\}}\mathbf{w}_k^{\{2\}}$$

$$z_k^{\{2\}} = h(a_k^{\{2\}})$$

- $y$ – output unit:

$$y = \sum_{k=1}^{K_2=2} w_k^{\{o\}} z_k^{\{2\}} = \mathbf{z}^{\{2\}}\mathbf{w}^{\{o\}}$$




Pop quiz: What's superscript and subscript for $w$'s from input $\mathbf{x}$ to hidden unit $z_3^{\{1\}}$?

# Notations of a 2-layer MLP: layer 2

- $z_j^{\{1\}}$ – hidden units $j \in \{1, 2, 3\}$ at layer 1:

$$a_j^{\{1\}} = \sum_{d=1}^{D=4} w_{d,j}^{\{1\}} x_d = \mathbf{x}\mathbf{w}_j^{\{1\}}$$

$$z_j^{\{1\}} = h(a_j^{\{1\}})$$

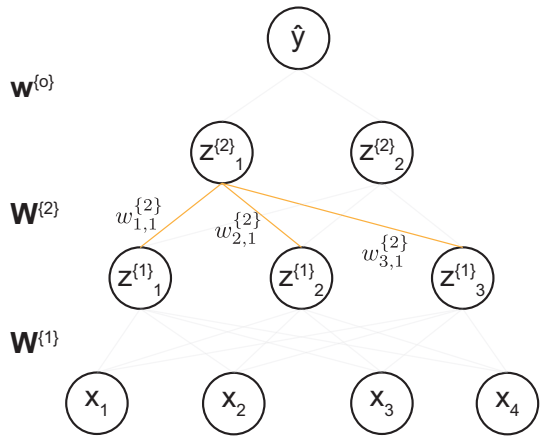
- $z_k^{\{2\}}$ – hidden units $k \in \{1, 2\}$ at layer 2:

$$a_k^{\{2\}} = \sum_{j=1}^{K_1=3} w_{j,k}^{\{2\}} z_j^{\{1\}} = \mathbf{z}^{\{1\}}\mathbf{w}_k^{\{2\}}$$

$$z_k^{\{2\}} = h(a_k^{\{2\}})$$

- $\hat{y}$ – output unit:

$$\hat{y} = \sum_{k=1}^{K_2=2} w_k^{\{o\}} z_k^{\{2\}} = \mathbf{z}^{\{2\}}\mathbf{w}^{\{o\}}$$




Pop quiz: What's superscript and subscript for $w$'s from hidden $\mathbf{z}^{\{1\}}$ to hidden unit $z_2^{\{2\}}$?

# Notations of a 2-layer MLP: output layer

- $z_j^{\{1\}}$ – hidden units $j \in \{1, 2, 3\}$ at layer 1:

$$a_j^{\{1\}} = \sum_{d=1}^{D=4} w_{d,j}^{\{1\}} x_d = \mathbf{x}\mathbf{w}_j^{\{1\}}$$

$$z_j^{\{1\}} = h(a_j^{\{1\}})$$

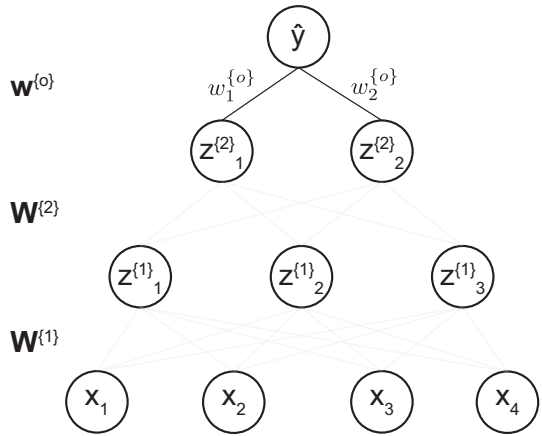
- $z_k^{\{2\}}$ – hidden units $k \in \{1, 2\}$ at layer 2:

$$a_k^{\{2\}} = \sum_{j=1}^{K_1=3} w_{j,k}^{\{2\}} z_j^{\{1\}} = \mathbf{z}^{\{1\}}\mathbf{w}_k^{\{2\}}$$

$$z_k^{\{2\}} = h(a_k^{\{2\}})$$

- $\hat{y}$ – output unit:

$$\hat{y} = \sum_{k=1}^{K_2=2} w_k^{\{o\}} z_k^{\{2\}} = \mathbf{z}^{\{2\}}\mathbf{w}^{\{o\}}$$



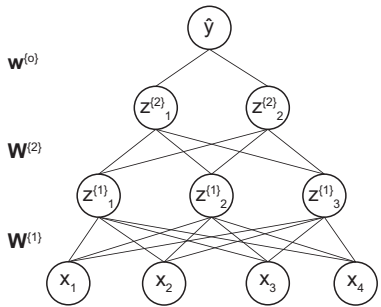

Pop quiz: What are the dimensions for $\mathbf{W}^{\{1\}}$, $\mathbf{W}^{\{2\}}$, and $\mathbf{w}^{\{o\}}$?

# 2-layer MLP for classification

Sigmoid output for binary classification:

$$a^{\{o\}} = \mathbf{z}^{\{2\}}\mathbf{w}^{\{o\}}$$

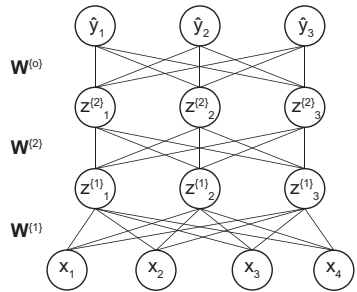
$$\hat{y} = \frac{1}{1+\exp(-a^{\{o\}})} = \sigma(a^{\{o\}})$$



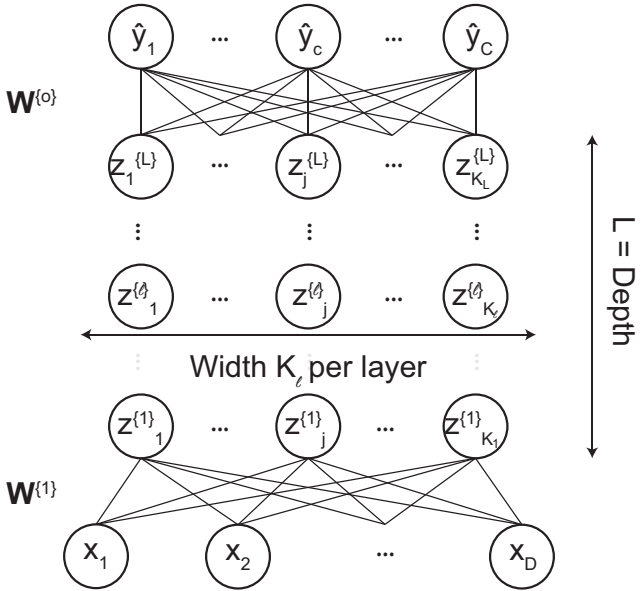

Softmax output for multiclass prediction:

$$\mathbf{a}^{\{o\}} = \mathbf{z}^{\{2\}}\mathbf{W}^{\{o\}}$$

$$\hat{\mathbf{y}} = \frac{\exp(\mathbf{a}^{\{o\}})}{\sum_c \exp(a_c^{\{o\}})}$$



# $L$-layer MLP


$\hat{y}_1$ ... $\hat{y}_c$ ... $\hat{y}_C$
$\mathbf{W}^{\{o\}}$
$z_1^{\{L\}}$ ... $z_j^{\{L\}}$ ... $z_{K_L}^{\{L\}}$
$z^{\{\ell\}}_1$ ... $z^{\{\ell\}}_j$ ... $z^{\{\ell\}}_{K_\ell}$
Width $K_\ell$ per layer
L = Depth
$z^{\{1\}}_1$ ... $z^{\{1\}}_j$ ... $z^{\{1\}}_{K_1}$
$\mathbf{W}^{\{1\}}$
$x_1$ $x_2$ ... $x_D$

# General and matrix notations of $L$-layer MLP

- $\mathbf{z}^{\{1\}}$ – hidden units at layer $\ell = 1$:

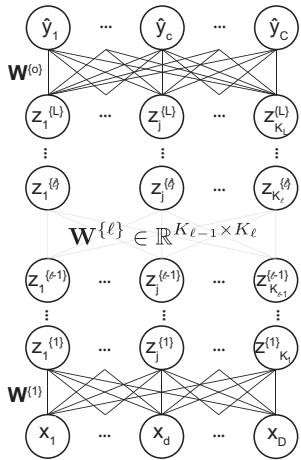
$$\mathbf{a}^{\{1\}} = \sum_{d=1}^{D} x_d \mathbf{w}_{d,\cdot}^{\{1\}} = \mathbf{x}\mathbf{W}^{\{1\}}$$
$$\mathbf{z}^{\{1\}} = h(\mathbf{a}^{\{1\}})$$

- $\mathbf{z}^{\{\ell\}}$ – hidden units at layer $\ell > 1$:

$$\mathbf{a}^{\{\ell\}} = \sum_{j=1}^{K_{\ell-1}} z_j^{\{\ell-1\}} \mathbf{w}_{j,\cdot}^{\{\ell\}} = \mathbf{z}^{\{\ell-1\}}\mathbf{W}^{\{\ell\}}$$
$$\mathbf{z}^{\{\ell\}} = h(\mathbf{a}^{\{\ell\}})$$

- $\hat{\mathbf{y}}$ – output units at the output layer:

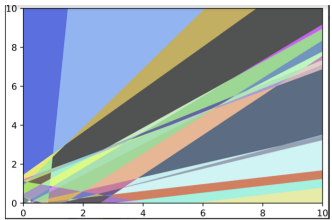
$$\hat{\mathbf{y}} = \phi(\sum_{k=1}^{K_L} z_k^{\{L\}} \mathbf{w}_{k,\cdot}^{\{o\}}) = \phi(\mathbf{z}^{\{L\}}\mathbf{W}^{\{o\}})$$



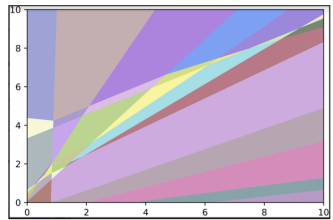

Note: $\mathbf{W}^{\{\ell\}}$ is a matrix if both layer $\ell - 1$ and layer $\ell$ have more than one input or hidden unit.

# Expressive power of a Deep Neural Network (DNN)

- An MLP with one hidden layer of enough hidden units is a universal function approximator, meaning it can model any function to any desired accuracy [Hor91]
- Sufficiently large combination of hidden units can "carve up" any region of space
- But it is shown that **deep** networks (i.e., large L) work better than shallow ones.
- Later layers can leverage the features that are learned by earlier layers; that is, the function is defined in a compositional or hierarchical way (e.g., XOR-MLP).



one-layer of 25 hidden



two-layers of 25 hidden each

K. Hornik. Approximation Capabilities of Multilayer Feedforward Networks. In: Neural Networks 4.2 (1991), pp. 251–257.

# Outline

# Activation functions

- The Heaviside activation function $f(\mathbf{x}) \in \{0, 1\}$ is not differentiable. We can replace it with a differentiable activation function $f(\mathbf{x}) \in \mathbb{R}$.
- The simplest and naïve activation function is *linear activation function*. But if we use a series of linear projection, our MLP reduces to a regular linear model:

$$
\begin{aligned}
& f(x; \mathbf{w}^{\{1\}}, \mathbf{w}^{\{2\}}, \ldots, \mathbf{w}^{\{L\}}) \\
& = f_L(f_{L-1}(\ldots f_3(f_2(f_1(\mathbf{x}\mathbf{W}^{\{1\}})\mathbf{W}^{\{2\}})\mathbf{W}^{\{3\}}) \ldots \mathbf{W}^{\{L-1\}})\mathbf{W}^{\{L\}}) \\
& \equiv \mathbf{x}\mathbf{W}^{\{1\}}\mathbf{W}^{\{2\}}, \ldots, \mathbf{W}^{\{L\}} \\
& = \mathbf{x}\mathbf{W}^{*}
\end{aligned}
$$

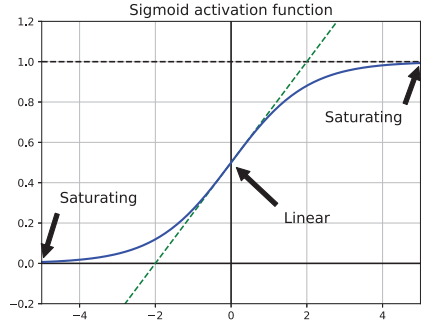
  where $\mathbf{W}^{*} = \mathbf{W}^{\{1\}}\mathbf{W}^{\{2\}}, \ldots, \mathbf{W}^{\{L\}}$
- Therefore, we need to consider *differentiable* AND *nonlinear* activation functions.

# Sigmoid as a non-linear activation function

Early neural networks use sigmoid as the activation function:

$$z = \frac{1}{1 + \exp(-a)}$$

- Pros:
  - a smooth approximation of Heaviside
  - Differentiable and has nice derivative $\frac{\partial z}{\partial a} = z(1 - z)$
- Cons:
  - Sigmoid **saturates** at 1 for large positive and 0 for large negative inputs leading to zero gradient: $\frac{\partial z}{\partial a} = z(1 - z) \approx 0$
  - For deep neural networks (i.e., DNN with large $L$) this causes **vanishing gradient** problem when backpropagating the partial derivative (details in Module 5.2):



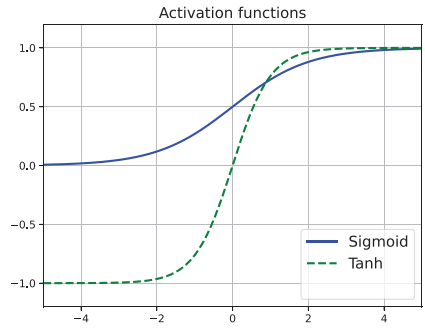

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{\{L\}}} \frac{\partial z^{\{L\}}}{\partial a^{\{L\}}} \frac{\partial a^{\{L\}}}{\partial z^{\{L-1\}}} \frac{\partial z^{\{L-1\}}}{\partial a^{\{L-1\}}} \frac{\partial a^{\{L-1\}}}{\partial z^{\{L-2\}}} \cdots \frac{\partial z^{\{\ell+1\}}}{\partial a^{\{\ell+1\}}} \frac{\partial a^{\{\ell+1\}}}{\partial z^{\{\ell\}}} \frac{\partial z^{\{\ell\}}}{\partial a^{\{\ell\}}} \frac{\partial a^{\{\ell\}}}{\partial w^{\{\ell\}}} \approx 0$$

# Hyperbolic tangent as another non-linear activation function

Hyperbolic tangent function or *Tanh* often used in recurrent neural networks (Module 5.4) behaves similarly and saturates at 1 and -1 for large positive and negative inputs.

$$z = \tanh(a) = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)}$$

$$\begin{aligned}\frac{\partial z}{\partial a} &= \left(\frac{\partial z}{\partial a}(\exp(a) - \exp(-a))(\exp(a) + \exp(-a))^{-1}\right) \\ &+ (\exp(a) - \exp(-a))\left(\frac{\partial z}{\partial a}(\exp(a) + \exp(-a))^{-1}\right) \\ &= \frac{\exp(a) + \exp(-a)}{\exp(a) + \exp(-a)} + \frac{(\exp(a) - \exp(-a))^2}{(\exp(a) + \exp(-a))^2} \\ &= 1 - \tanh^2(a) \in [0, 1]\end{aligned}$$




By multiplying a lot of decimal values, the gradient can rapidly vanish:

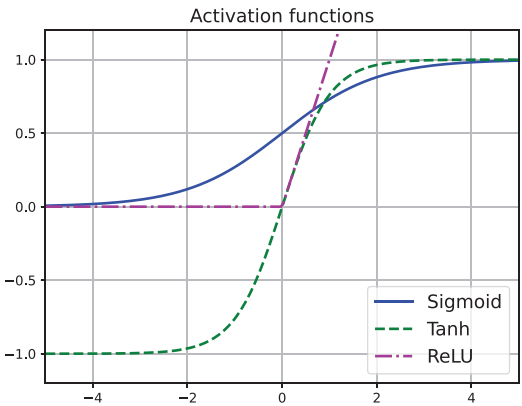
$$\frac{\partial L}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial z^{\{L\}}}\frac{\partial z^{\{L\}}}{\partial a^{\{L\}}}\frac{\partial a^{\{L\}}}{\partial z^{\{L-1\}}}\frac{\partial z^{\{L-1\}}}{\partial a^{\{L-1\}}}\frac{\partial a^{\{L-1\}}}{\partial z^{\{L-2\}}}\cdots\frac{\partial z^{\{\ell+1\}}}{\partial a^{\{\ell+1\}}}\frac{\partial a^{\{\ell+1\}}}{\partial z^{\{\ell\}}}\frac{\partial z^{\{\ell\}}}{\partial a^{\{\ell\}}}\frac{\partial a^{\{\ell\}}}{\partial w^{\{\ell\}}} \approx 0$$

# Rectified linear unit (ReLU) – the most common activation function

One key to effectively train a deep neural networks (DNN) is to use a non-saturating activation function. The ReLU is a common activation function used at practice:

$$z = ReLU(a) = \max(a, 0)$$

ReLU function simply "turns off" negative inputs and passes positive inputs *unchanged*.



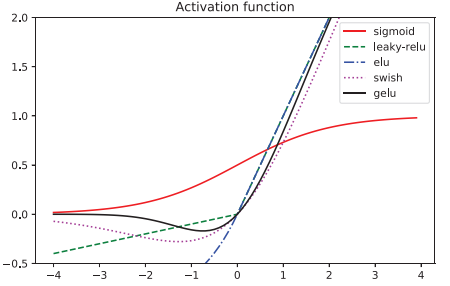

Pop quiz: what's the derivative of

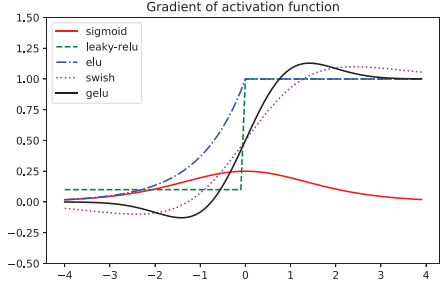
$$\frac{\partial z}{\partial a} = ?$$

# Common activation functions (Murphy22 13.4.3)

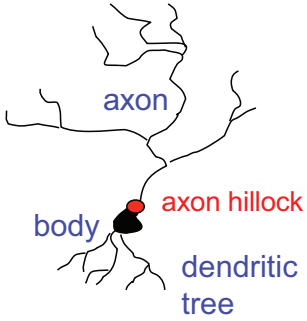
| Name | Definition | Range | Reference |
|---|---|---|---|
| Sigmoid | $\sigma(a) = \frac{1}{1+e^{-a}}$ | $[0, 1]$ | |
| Hyperbolic tangent | $\tanh(a) = 2\,\sigma(2a) - 1$ | $[-1, 1]$ | |
| Softplus | $\sigma_+(a) = \log(1 + e^a)$ | $[0, \infty]$ | [GBB11] |
| Rectified linear unit | $\mathrm{ReLU}(a) = \max(a, 0)$ | $[0, \infty]$ | [GBB11; KSH12] |
| Leaky ReLU | $\max(a, 0) + \alpha \min(a, 0)$ | $[-\infty, \infty]$ | [MHN13] |
| Exponential linear unit | $\max(a, 0) + \min(\alpha(e^a - 1), 0)$ | $[-\infty, \infty]$ | [CUH16] |
| Swish | $a\sigma(a)$ | $[-\infty, \infty]$ | [RZL17] |
| GELU | $a\Phi(a)$ | $[-\infty, \infty]$ | [HG16] |

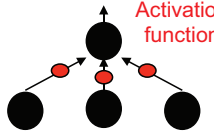
Can you compute derivative w.r.t. a for all of the above activation functions?



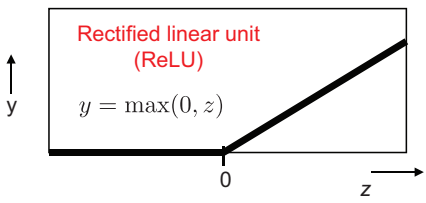
# How does brain work?

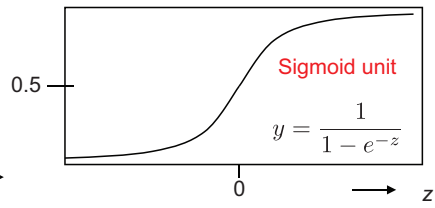



$$z = b + \sum_i x_i w_i$$

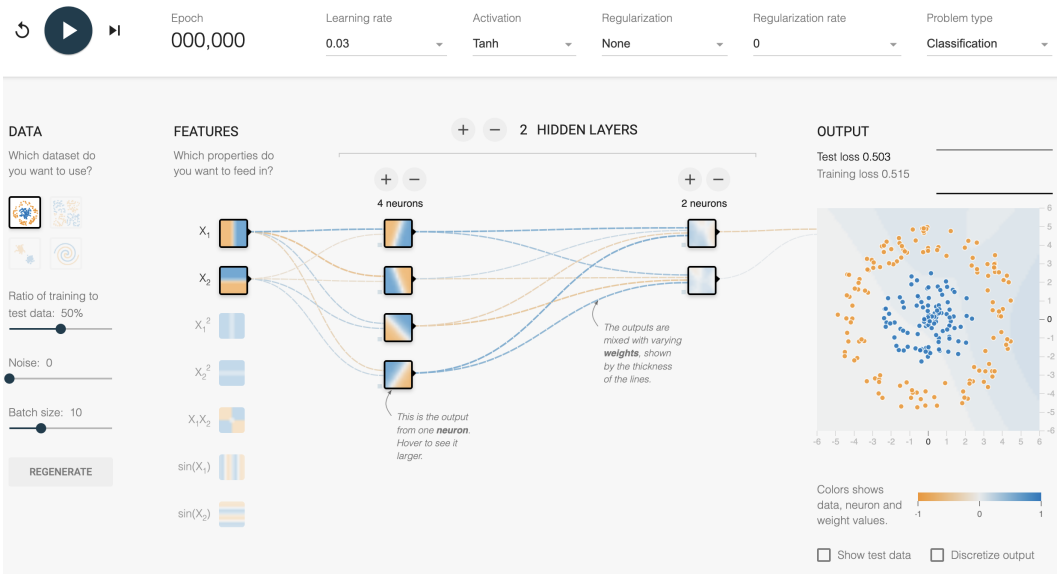
Popular functions:



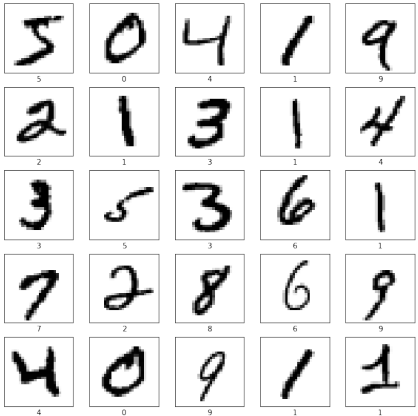
# Outline

# Toy data: interactive visualization of 2D data classification

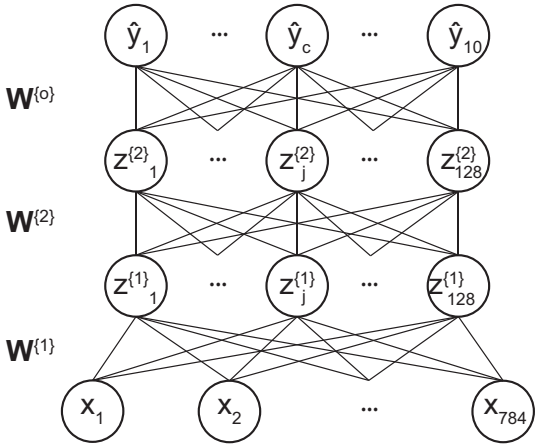


# MNIST classification (video)

25 out of 60K $28 \times 28$ MNIST images



Two-layer feedforward network trained on flatten images (784 input units)



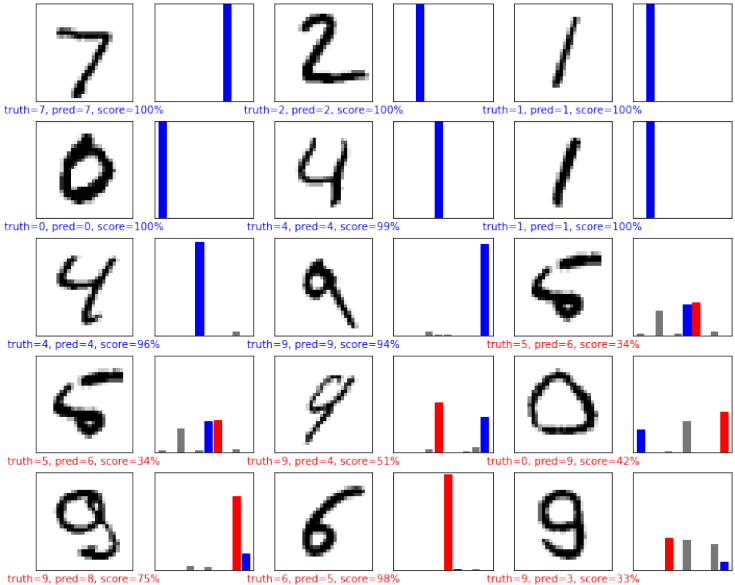
# Keras implementation (with Tensorflow as backend) (colab)

```
model = keras.Sequential(
    [
        keras.layers.Flatten(input_shape=(28, 28)),
        keras.layers.Dense(128, activation=tf.nn.relu),
        keras.layers.Dense(128, activation=tf.nn.relu),
        keras.layers.Dense(10, activation=tf.nn.softmax),
    ]
)
model.summary()
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
↪  metrics=["accuracy"])
time_start = time()
model.fit(train_images, train_labels, epochs=1)
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 784)               0
_________________________________________________________________
dense (Dense)                (None, 128)               100480
_________________________________________________________________
dense_1 (Dense)              (None, 128)               16512
_________________________________________________________________
dense_2 (Dense)              (None, 10)                1290
=================================================================
Total params: 118,282
```

- Trained for only 1 epoch (i.e., 1 iteration full pass over 1875 mini-batches of 32 images)
- Training/test accuracy: 98%/97%
- Pop quiz: How was the Param # computed?

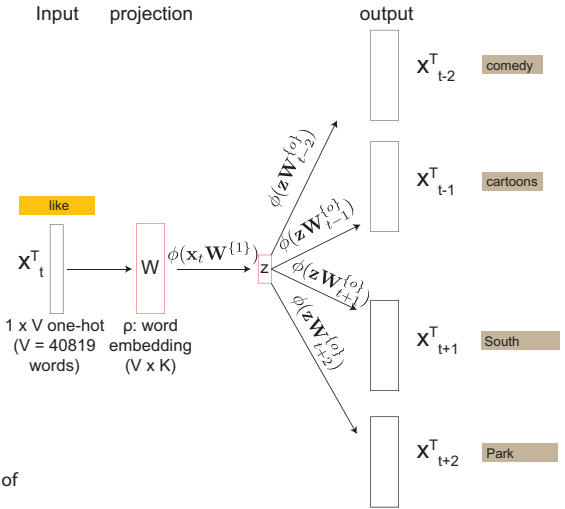
# Interesting test images (Red is incorrect, blue is correct)



# Text classification – Step 1. learn word embedding by skip-gram

IMDB movie review

If you like adult comedy cartoons, like South Park, then this is nearly a similar format about the small adventures of three teenage girls at Bromwell High. Keisha, Natella and Latrina have given exploding sweets and behaved like bitches, I think Keisha is a good leader. There are also small stories going on with the teachers of the school. There's the idiotic principal, Mr. Bip, the nervous Maths teacher and many others. The cast is also fantastic, Lenny Henry's Gina Yashere, EastEnders Chrissie Watts, Tracy-Ann Oberman, Smack The Pony's Doon Mackichan, Dead Ringers' Mark Perry and Blunder's Nina Conti. I didn't know this came from Canada, but it is very good. Very good!




Mikolov, T., Chen, K., Corrado, G. & Dean, J. Efficient Estimation of Word Representations in Vector Space. Arxiv (2013).

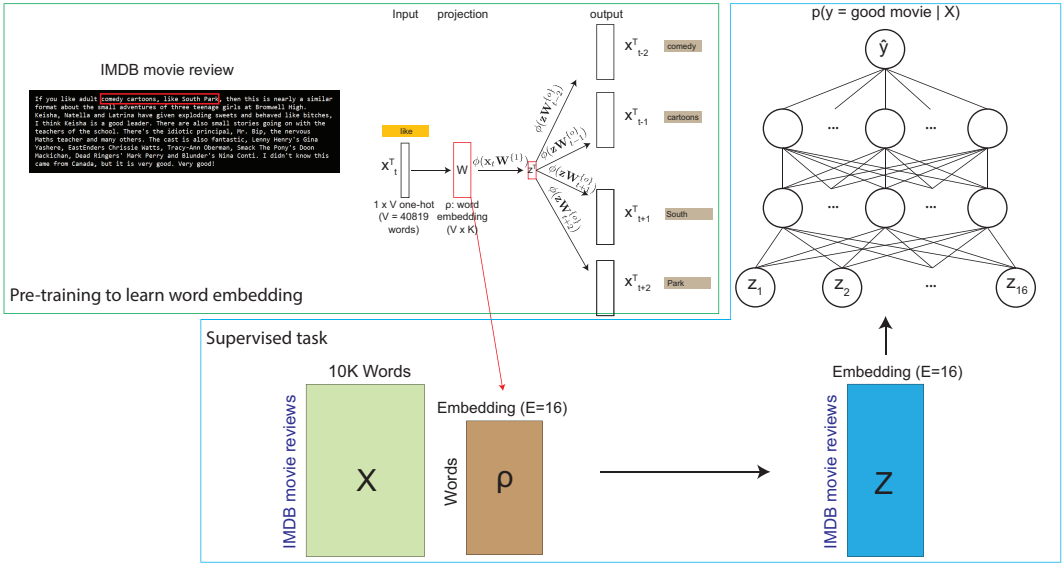
# Skip-gram's word embedding accurately recovers word relations [Mik+13]

- The word relation is defined by subtracting two word vectors, and the result is added to another word.
- For example, *Paris* - *France* + *Italy* = *Rome*.

Table 8: Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).

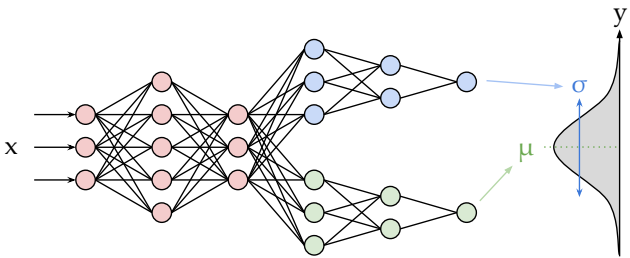
| Query Relation | Example 1 | Example 2 | Example 3 |
|---|---|---|---|
| France : Paris | Italy: Rome | Japan: Tokyo | Florida: Tallahassee |
| big : bigger | small: larger | cold: colder | quick: quicker |
| Miami : Florida | Baltimore: Maryland | Dallas: Texas | Kona: Hawaii |
| Einstein : scientist | Messi: midfielder | Mozart: violinist | Picasso: painter |
| Sarkozy : France | Berlusconi: Italy | Merkel: Germany | Koizumi: Japan |
| copper : Cu | zinc: Zn | gold: Au | uranium: plutonium |
| Berlusconi : Silvio | Sarkozy: Nicolas | Putin: Medvedev | Obama: Barack |
| Microsoft : Windows | Google: Android | IBM: Linux | Apple: iPhone |
| Microsoft : Ballmer | Google: Yahoo | IBM: McNealy | Apple: Jobs |
| Japan : sushi | Germany: bratwurst | France: tapas | USA: pizza |

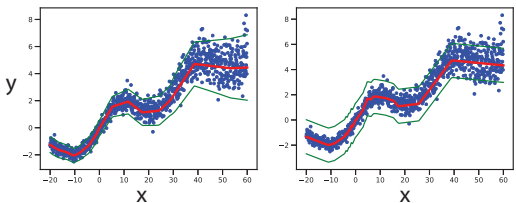
# Text classification – Step 2. MLP training (colab)



# Heteroskedastic regression: variance-mean dependent regression (source)

$$p(y|\mathbf{x}, \theta) = \mathcal{N}(y; f_\mu(f(\mathbf{x}; \mathbf{W}_{shared}); \mathbf{W}_\mu), f_\sigma(f(\mathbf{x}; \mathbf{W}_{shared}); \mathbf{W}_\sigma)$$

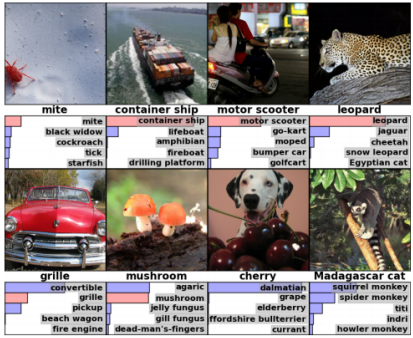


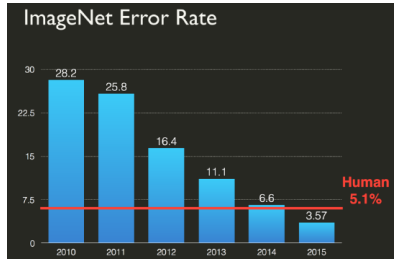




- MLP with the first two layers shared and two output heads, one for predicting the mean and one for predicting the variance
- The bottom left plot illustrates 1D prediction using the above MLP. y-axis is the predicted mean and the green curves indicate the predicted variance
- The bottom right plot shows a model that predicts only the mean. The variance is estimated as a constant $\sigma^2$ for a training data points via maximum likelihood estimation.

# The "deep learning revolution"

- Automatic speech recognition based on breakthrough results in [Hin+12]
- AlexNet reduced the error rate from 25.8% to 16.4% in a single year in the ImageNet challenge using a deep ConvNet (Module 5.3) [Kriz+12]



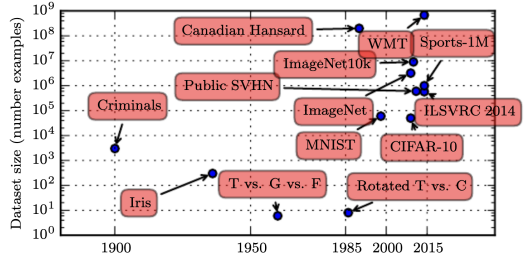

[Hin+12] Hinton, G. et al. Deep neural networks for acoustic modeling in speech recognition. IEEE Signal Processing Magazine 29, 82–97 (2012).
[Kriz+12] Krizhevsky, Alex; Sutskever, Ilya; Hinton, Geoffrey E. ImageNet classification with deep convolutional neural networks. NeurIPS Proceedings 2012

# Contributing factors to the DNN revolution

1. Big Data. For example, ImageNet was trained on 1.3M labelled images with millions of parameters.

   Andrew Ng: "*If deep learning systems are viewed as "rockets", then large datasets have been called the fuel*"
2. Faster computers thanks to Graphics processing units (GPU).
3. High-quality open-source software, e.g., Tensorflow, PyTorch, etc.
4. However, all above would not happen without curiosity-driven research of how brain works: the "godfather" of deep learning Geoffrey Hinton is an experimental psychologist.

   book: Genius Makers by Cade Metz




Source: Figure 1.11

# Summary

- Perceptron is limited in its linear decision boundary
- MLP or often known as neural networks is composed of layers of adaptive nonlinear basis functions called hidden units with learnable weights
- Increasing depth (i.e., number of hidden layers) is often preferable to width (i.e., number of hidden units)
- Various choices of activation functions are important to realize the universal approximation prowess of neural networks
- Deep neural networks (DNN) are branded as Deep Learning
- Besides the excitements in what they can do, GPU, data, and high quality API are the main contributing factors to the popularity of DNN
- Optional: if you are interested in the recent history with interesting narratives by someone renown in the field, here is a start:
  - Deep learning revolution (2018) Terrence J. Sejnowski, Professor at the Salk Institute
  - The Worlds I See (2023) by Feifei Li, Professor at Stanford University